
```
Traceback (most recent call last):
  File "zeroDivideTraceback.py", line 4, in <module>
    spam(101, 42)
  File "zeroDivideTraceback.py", line 2, in spam
    return number1 / (number2 - 42)
ZeroDivisionError: division by zero
```

The error message is the same, but the zero divide in `return number1 / (number2 - 42)` isn't quite so obvious. You can deduce that there is a division happening from the `/` operator, and that the expression `(number2 - 42)` must evaluate to `0`. This would lead you to conclude that the `spam()` function fails whenever the `number2` parameter is set to `42`.

Sometimes the traceback might indicate that an error is on the line after the true cause of the bug. For example, in the following program, the first line is missing the closing parenthesis:

```
print('Hello.'
print('How are you?')
```

But the error message for this program indicates the problem is on the second line:

```
File "example.py", line 2
    print('How are you?')
    ^
SyntaxError: invalid syntax
```

The reason is that the Python interpreter didn't notice the syntax error until it read the second line. The traceback can indicate where things went wrong, but that isn't always the same as where the actual cause of a bug is. If the frame summary doesn't give you enough information to figure out the bug, or if the true cause of the bug is on a previous line not shown by the traceback, you'll have to step through the program with a debugger or check any logging messages to find the cause. This can take a significant amount of time. An internet search of the error message might give you critical clues about the solution much more quickly.

Searching for Error Messages